

ATmega64 Morse Code Generator

Microprocessor Project Report

Author:

Farham Asgari

=====

1. Introduction

Morse code is a communication method in which letters and symbols are represented using sequences of dots and dashes.

The objective of this project is to design and implement a Morse code generator using an ATmega64 microcontroller and AVR Assembly language.

The generated Morse sequence is displayed using LED blinking patterns and waveform visualization inside Proteus.

=====

2. Project Objectives

The main objectives of the project are:

- Morse code generation

- User-selectable character inputs
- Adjustable timing modes
- LED signal generation
- Continuous message repetition
- Accurate timer-based delays

=====

3. Hardware Requirements

The hardware components used include:

- ATmega64 Microcontroller
- LED
- Current limiting resistor
- Input switches
- Proteus simulation environment

4. Software Requirements

The software implementation includes:

- AVR Assembly Language
 - Timer0 configuration
 - Register manipulation
 - Port management
-

5. System Design

The implementation follows several stages:

Stage 1:

Initialize ports and registers

Stage 2:

Read user inputs

Stage 3:

Determine character identifiers

Stage 4:

Determine timing mode

Stage 5:

Generate Morse sequence

Stage 6:

Generate output signal

Stage 7:

Repeat message

=====

6. Input System

Character selection is implemented using multiple microcontroller ports.

Character identifiers are stored inside:

R17

R18

R19

R29

R28

R27

Timing mode selection uses:

PC3–PC0

=====

7. Morse Code Implementation

Each character is assigned a unique binary identifier.

The system compares incoming identifiers with stored patterns.

After identification:

- Dot function executes
- Dash function executes
- Delays are inserted

=====

8. Timing System

Timer0 is used for generating delays.

Delay categories:

- Dot timing
- Dash timing
- Inter-element timing
- Character timing

- Word timing

Maintaining constant ratios ensures Morse readability.

=====

9. Results

The system successfully achieved:

- Morse code generation
- Adjustable speed modes
- Waveform visualization
- LED blinking output
- Continuous message repetition

=====

10. Challenges

Challenges during implementation:

- Limited hardware resources
- Timing synchronization
- Dynamic input limitations
- Register management complexity

=====

11. Learning Outcomes

The project provided practical experience in:

- AVR Assembly Programming
- Embedded Systems Design
- Hardware/Software Integration
- Timer Configuration
- State-based implementation
- Low-level programming

12. Future Improvements

Potential enhancements:

- Dynamic runtime input
- LCD support
- UART communication
- Audio Morse generation
- External keyboard support

13. Conclusion

The project successfully implemented a complete Morse code generation system using ATmega64 and AVR Assembly.

The final implementation demonstrates how embedded systems can combine hardware and software modules to create reliable real-time communication systems.